

BITÁCORA DE IA

PARTE 1

Prompt utilizado: "Actúa como un arquitecto de software senior. Necesito la estructura básica en Java para un programa llamado p080_SimuladorFinanciero. Crea los esqueletos de tres funciones estáticas: calcularAhorro, calcularInteresCompuesto y obtenerPerfil. Incluye solo los comentarios Javadoc de lo que debe hacer cada una y la estructura de parámetros. No escribas el código interno todavía."

Lo que entregó la ia:

```
public class p080_SimuladorFinanciero {

    /**
     * Calcula el monto total de ahorro acumulado tras un número determinado
     * de períodos, dado un depósito inicial y una aportación periódica.
     *
     * @param montoInicial Cantidad de dinero con la que se inicia el ahorro (debe ser >=
    0).
     * @param aportacionPeriodica Cantidad de dinero que se añade en cada período (debe
    ser >= 0).
     * @param numeroPeriodos Número de períodos durante los que se realizará el ahorro
    (debe ser > 0).
     * @return El monto total ahorrado al finalizar los períodos indicados.
     */
    public static double calcularAhorro(double montoInicial, double aportacionPeriodica, int
    numeroPeriodos) {
        // TODO: implementar lógica de cálculo de ahorro
        return 0.0;
    }

    /**
     * Calcula el monto final de una inversión aplicando interés compuesto
     * sobre un capital inicial durante un número de períodos a una tasa dada.
     *
     * @param capitalInicial Capital invertido al inicio del período (debe ser >= 0).
     * @param tasaInteres Tasa de interés por período, expresada en decimal (ej. 0.05 =
    5%).
     * @param numeroPeriodos Número de períodos que durará la inversión (debe ser > 0).
     * @return El monto final obtenido tras aplicar el interés compuesto.
     */
    public static double calcularInteresCompuesto(double capitalInicial, double tasaInteres, int
    numeroPeriodos) {
        // TODO: implementar lógica de interés compuesto
        return 0.0;
    }
}
```

```
/**
 * Determina el perfil financiero del usuario a partir de su nivel de ingresos
 * y su nivel de endeudamiento, devolviendo una categoría descriptiva.
 *
 * @param ingresosMensuales Ingreso mensual neto del usuario (debe ser >= 0).
 * @param nivelEndeudamiento Porcentaje de endeudamiento sobre ingresos, en
 * decimal (ej. 0.30 = 30%).
 * @return Una cadena con el perfil financiero resultante (ej. "Conservador", "Moderado",
 * "Agresivo").
 */
public static String obtenerPerfil(double ingresosMensuales, double nivelEndeudamiento)
{
    // TODO: implementar lógica de clasificación de perfil
    return "";
}

public static void main(String[] args) {
    // TODO: punto de entrada del simulador financiero
}
}
```

PARTE 2

Prompt utilizado: "Implementa la lógica de la función calcularAhorro en Java usando un ciclo for que acumule el interés mes a mes. También implementa calcularInteresCompuesto usando Math.pow. Por último, crea obtenerPerfil que clasifique según: < 30 años y monto > 10k es Agresivo; > 50 años es Conservador; el resto es Moderado. Incluye validaciones para que los números sean positivos."

Lo que entregó la ia:

```
public class p080_SimuladorFinanciero {

    /**
     * Calcula el monto total de ahorro acumulado tras un número determinado
     * de períodos, dado un depósito inicial, una aportación periódica y una
     * tasa de interés mensual, acumulando el interés mes a mes.
     *
     * @param montoInicial Cantidad de dinero con la que se inicia el ahorro (debe ser
     * >= 0).
     * @param aportacionPeriodica Cantidad de dinero que se añade en cada período (debe
     * ser >= 0).
     * @param tasaInteresMensual Tasa de interés mensual en decimal (ej. 0.01 = 1%).
     * Debe ser >= 0.
     */
}
```

Alumno: Emiliano Martinez Torres

```
* @param numeroPeriodos    Número de períodos (meses) durante los que se ahorrará
(debe ser > 0).
* @return El monto total ahorrado al finalizar los períodos indicados.
* @throws IllegalArgumentException si algún parámetro es negativo o numeroPeriodos
<= 0.
*/
public static double calcularAhorro(double montoInicial,
                                   double aportacionPeriodica,
                                   double tasaInteresMensual,
                                   int numeroPeriodos) {

    if (montoInicial < 0) {
        throw new IllegalArgumentException("El monto inicial no puede ser negativo.");
    }
    if (aportacionPeriodica < 0) {
        throw new IllegalArgumentException("La aportación periódica no puede ser
negativa.");
    }
    if (tasaInteresMensual < 0) {
        throw new IllegalArgumentException("La tasa de interés no puede ser negativa.");
    }
    if (numeroPeriodos <= 0) {
        throw new IllegalArgumentException("El número de períodos debe ser mayor que
cero.");
    }

    double saldo = montoInicial;

    for (int mes = 1; mes <= numeroPeriodos; mes++) {
        // Se aplica el interés sobre el saldo actual y luego se añade la aportación del mes.
        saldo = saldo * (1 + tasaInteresMensual) + aportacionPeriodica;
    }

    return saldo;
}

/**
 * Calcula el monto final de una inversión aplicando interés compuesto
 * sobre un capital inicial durante un número de períodos a una tasa dada.
 * Fórmula:  $M = C * (1 + i)^n$ 
 *
 * @param capitalInicial Capital invertido al inicio del período (debe ser > 0).
 * @param tasaInteres    Tasa de interés por período, expresada en decimal (debe ser >
0).
 * @param numeroPeriodos Número de períodos que durará la inversión (debe ser > 0).
 * @return El monto final obtenido tras aplicar el interés compuesto.
 * @throws IllegalArgumentException si algún parámetro no es positivo.
 */
```

```
public static double calcularInteresCompuesto(double capitalInicial,
                                             double tasaInteres,
                                             int numeroPeriodos) {

    if (capitalInicial <= 0) {
        throw new IllegalArgumentException("El capital inicial debe ser mayor que cero.");
    }
    if (tasaInteres <= 0) {
        throw new IllegalArgumentException("La tasa de interés debe ser mayor que cero.");
    }
    if (numeroPeriodos <= 0) {
        throw new IllegalArgumentException("El número de períodos debe ser mayor que
cero.");
    }

    return capitalInicial * Math.pow(1 + tasaInteres, numeroPeriodos);
}

/**
 * Determina el perfil financiero del usuario según su edad y el monto
 * de inversión disponible.
 * Reglas:
 * - Edad < 30 y monto > 10,000 -> "Agresivo"
 * - Edad > 50 -> "Conservador"
 * - En cualquier otro caso -> "Moderado"
 *
 * @param edad Edad del usuario en años (debe ser > 0).
 * @param monto Monto de inversión disponible (debe ser >= 0).
 * @return Una cadena con el perfil financiero resultante.
 * @throws IllegalArgumentException si la edad no es positiva o el monto es negativo.
 */
public static String obtenerPerfil(int edad, double monto) {

    if (edad <= 0) {
        throw new IllegalArgumentException("La edad debe ser mayor que cero.");
    }
    if (monto < 0) {
        throw new IllegalArgumentException("El monto no puede ser negativo.");
    }

    if (edad < 30 && monto > 10_000) {
        return "Agresivo";
    } else if (edad > 50) {
        return "Conservador";
    } else {
        return "Moderado";
    }
}
```

```
public static void main(String[] args) {
    // Ejemplos rápidos de uso
    double ahorro = calcularAhorro(1000, 200, 0.01, 12);
    System.out.printf("Ahorro tras 12 meses: %.2f%n", ahorro);

    double interes = calcularInteresCompuesto(10000, 0.05, 10);
    System.out.printf("Interés compuesto: %.2f%n", interes);

    System.out.println("Perfil (25 años, 15k): " + obtenerPerfil(25, 15000));
    System.out.println("Perfil (55 años, 5k): " + obtenerPerfil(55, 5000));
    System.out.println("Perfil (40 años, 8k): " + obtenerPerfil(40, 8000));
}
}
```

PARTE 3

Prompt utilizado: "Ahora, crea el método main para mi programa p080_SimuladorFinanciero. Debe usar un ciclo do-while y un switch-case para mostrar un menú con 4 opciones. Asegúrate de manejar correctamente el Scanner y limpiar el buffer si es necesario al leer strings después de números."

Lo que entregó la ia:

```
import java.util.Scanner;

public class p080_SimuladorFinanciero {

    // ... (métodos calcularAhorro, calcularInteresCompuesto y obtenerPerfil ya
    implementados)

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int opcion;

        do {
            System.out.println("\n===== SIMULADOR FINANCIERO =====");
            System.out.println("1. Calcular ahorro con aportaciones periódicas");
            System.out.println("2. Calcular interés compuesto");
            System.out.println("3. Obtener perfil financiero");
            System.out.println("4. Salir");
            System.out.print("Selecciona una opción: ");

            // Validación: asegura que el usuario ingrese un número entero
            while (!scanner.hasNextInt()) {
                System.out.print("Opción inválida. Ingrese un número del 1 al 4: ");
                scanner.next(); // descarta la entrada incorrecta
            }
        }
```

```
opcion = scanner.nextInt();
scanner.nextLine(); // limpia el buffer tras leer el entero

switch (opcion) {
    case 1: {
        System.out.println("\n--- Cálculo de Ahorro ---");
        double montoInicial = leerDouble(scanner, "Monto inicial: ");
        double aportacion = leerDouble(scanner, "Aportación mensual: ");
        double tasa = leerDouble(scanner, "Tasa de interés mensual (ej. 0.01 = 1%): ");
        int periodos = leerEnteroPositivo(scanner, "Número de meses: ");

        double resultado = calcularAhorro(montoInicial, aportacion, tasa, periodos);
        System.out.printf("Ahorro total tras %d meses: %.2f%n", periodos, resultado);
        break;
    }
    case 2: {
        System.out.println("\n--- Interés Compuesto ---");
        double capital = leerDouble(scanner, "Capital inicial: ");
        double tasa = leerDouble(scanner, "Tasa de interés por período (ej. 0.05 = 5%): ");
        int periodos = leerEnteroPositivo(scanner, "Número de períodos: ");

        double resultado = calcularInteresCompuesto(capital, tasa, periodos);
        System.out.printf("Monto final: %.2f%n", resultado);
        break;
    }
    case 3: {
        System.out.println("\n--- Perfil Financiero ---");
        int edad = leerEnteroPositivo(scanner, "Edad: ");
        double monto = leerDouble(scanner, "Monto de inversión disponible: ");

        String perfil = obtenerPerfil(edad, monto);
        System.out.println("Perfil financiero: " + perfil);
        break;
    }
    case 4:
        System.out.println("\nGracias por usar el Simulador Financiero. ¡Hasta pronto!");
        break;
    default:
        System.out.println("\nOpción no válida. Intenta de nuevo.");
}

} while (opcion != 4);

scanner.close();
}
```

```
/**
 * Solicita al usuario un número decimal por consola, validando que sea un double.
 */
private static double leerDouble(Scanner scanner, String mensaje) {
    System.out.print(mensaje);
    while (!scanner.hasNextDouble()) {
        System.out.print("Entrada inválida. " + mensaje);
        scanner.next();
    }
    double valor = scanner.nextDouble();
    scanner.nextLine(); // limpieza del buffer
    return valor;
}

/**
 * Solicita al usuario un entero positivo por consola.
 */
private static int leerEnteroPositivo(Scanner scanner, String mensaje) {
    System.out.print(mensaje);
    while (!scanner.hasNextInt()) {
        System.out.print("Entrada inválida. " + mensaje);
        scanner.next();
    }
    int valor = scanner.nextInt();
    scanner.nextLine(); // limpieza del buffer
    return valor;
}
}
```

PARTE 4

Prompt utilizado: "Revisa mi código completo de p080_SimuladorFinanciero. Busca posibles errores de lógica, asegura que las tasas de interés se manejen como decimales (ej. 5% como 0.05) y optimiza el formato de salida para que los montos de dinero muestren solo 2 decimales."

Lo que entregó la ia:

```
import java.util.Scanner;

public class p080_SimuladorFinanciero {

    // =====
    // 1) CÁLCULO DE AHORRO CON APORTACIONES PERIÓDICAS
    // =====
    /**
     * Calcula el monto total de ahorro acumulado tras un número determinado
     * de períodos, dado un depósito inicial, una aportación periódica y una
```

```
* tasa de interés mensual, acumulando el interés mes a mes.
*
* @param montoInicial Cantidad de dinero con la que se inicia el ahorro (>= 0).
* @param aportacionPeriodica Cantidad que se añade en cada período (>= 0).
* @param tasaInteresMensual Tasa mensual en decimal (ej. 0.01 = 1%). Debe ser >= 0.
* @param numeroPeriodos Número de períodos/meses a ahorrar (> 0).
* @return Monto total ahorrado al finalizar los períodos indicados.
* @throws IllegalArgumentException si algún parámetro es inválido.
*/
public static double calcularAhorro(double montoInicial,
                                   double aportacionPeriodica,
                                   double tasaInteresMensual,
                                   int numeroPeriodos) {

    if (montoInicial < 0) {
        throw new IllegalArgumentException("El monto inicial no puede ser negativo.");
    }
    if (aportacionPeriodica < 0) {
        throw new IllegalArgumentException("La aportación periódica no puede ser
negativa.");
    }
    if (tasaInteresMensual < 0) {
        throw new IllegalArgumentException("La tasa de interés no puede ser negativa.");
    }
    if (numeroPeriodos <= 0) {
        throw new IllegalArgumentException("El número de períodos debe ser mayor que
cero.");
    }

    double saldo = montoInicial;
    for (int mes = 1; mes <= numeroPeriodos; mes++) {
        // Interés sobre el saldo actual + aportación del mes (al final del mes)
        saldo = saldo * (1 + tasaInteresMensual) + aportacionPeriodica;
    }
    return saldo;
}

// =====
// 2) INTERÉS COMPUESTO
// =====
/**
 * Calcula el monto final aplicando interés compuesto:  $M = C * (1 + i)^n$ 
 *
 * @param capitalInicial Capital inicial (> 0).
 * @param tasaInteres Tasa por período en decimal (ej. 0.05 = 5%). Debe ser > 0.
 * @param numeroPeriodos Número de períodos (> 0).
 * @return Monto final tras aplicar interés compuesto.
 * @throws IllegalArgumentException si algún parámetro no es positivo.
 */
```

```
*/
public static double calcularInteresCompuesto(double capitalInicial,
                                              double tasaInteres,
                                              int numeroPeriodos) {

    if (capitalInicial <= 0) {
        throw new IllegalArgumentException("El capital inicial debe ser mayor que cero.");
    }
    if (tasaInteres <= 0) {
        throw new IllegalArgumentException("La tasa de interés debe ser mayor que cero.");
    }
    if (numeroPeriodos <= 0) {
        throw new IllegalArgumentException("El número de períodos debe ser mayor que
cero.");
    }

    return capitalInicial * Math.pow(1 + tasaInteres, numeroPeriodos);
}

// =====
// 3) PERFIL FINANCIERO
// =====
/**
 * Determina el perfil financiero según edad y monto de inversión:
 * - Edad < 30 y monto > 10,000 -> "Agresivo"
 * - Edad > 50 -> "Conservador"
 * - Resto -> "Moderado"
 *
 * @param edad Edad del usuario (> 0).
 * @param monto Monto de inversión disponible (>= 0).
 * @return Perfil financiero ("Agresivo", "Conservador" o "Moderado").
 * @throws IllegalArgumentException si la edad no es positiva o el monto es negativo.
 */
public static String obtenerPerfil(int edad, double monto) {

    if (edad <= 0) {
        throw new IllegalArgumentException("La edad debe ser mayor que cero.");
    }
    if (monto < 0) {
        throw new IllegalArgumentException("El monto no puede ser negativo.");
    }

    if (edad < 30 && monto > 10_000) {
        return "Agresivo";
    } else if (edad > 50) {
        return "Conservador";
    } else {
        return "Moderado";
    }
}
```

```
    }  
}  
  
// =====  
// 4) MAIN CON MENÚ INTERACTIVO  
// =====  
public static void main(String[] args) {  
    Scanner scanner = new Scanner(System.in);  
    int opcion;  
  
    do {  
        System.out.println("\n==== SIMULADOR FINANCIERO =====");  
        System.out.println("1. Calcular ahorro con aportaciones periódicas");  
        System.out.println("2. Calcular interés compuesto");  
        System.out.println("3. Obtener perfil financiero");  
        System.out.println("4. Salir");  
        System.out.print("Selecciona una opción: ");  
  
        // Validación: asegura que el usuario ingrese un entero  
        while (!scanner.hasNextInt()) {  
            System.out.print("Opción inválida. Ingresa un número del 1 al 4: ");  
            scanner.next();  
        }  
        opcion = scanner.nextInt();  
        scanner.nextLine(); // limpieza del buffer tras leer el entero  
  
        try {  
            switch (opcion) {  
                case 1: {  
                    System.out.println("\n--- Cálculo de Ahorro ---");  
                    double montoInicial = leerDoubleNoNegativo(scanner, "Monto inicial: ");  
                    double aportacion = leerDoubleNoNegativo(scanner, "Aportación mensual:  
");  
  
                    double tasa = leerTasaComoPorcentaje(scanner,  
                        "Tasa de interés mensual (en %, ej. 1 = 1%): ");  
                    int periodos = leerEnteroPositivo(scanner, "Número de meses: ");  
  
                    double resultado = calcularAhorro(montoInicial, aportacion, tasa, periodos);  
                    System.out.printf("Ahorro total tras %d meses: $%,.2f%n", periodos,  
resultado);  
  
                    break;  
                }  
                case 2: {  
                    System.out.println("\n--- Interés Compuesto ---");  
                    double capital = leerDoublePositivo(scanner, "Capital inicial: ");  
                    double tasa = leerTasaComoPorcentaje(scanner,  
                        "Tasa de interés por período (en %, ej. 5 = 5%): ");  
                    int periodos = leerEnteroPositivo(scanner, "Número de períodos: ");
```

```
        double resultado = calcularInteresCompuesto(capital, tasa, periodos);
        System.out.printf("Monto final: $%,.2f%n", resultado);
        break;
    }
    case 3: {
        System.out.println("\n--- Perfil Financiero ---");
        int edad = leerEnteroPositivo(scanner, "Edad: ");
        double monto = leerDoubleNoNegativo(scanner, "Monto de inversión
disponible: ");

        String perfil = obtenerPerfil(edad, monto);
        System.out.println("Perfil financiero: " + perfil);
        break;
    }
    case 4:
        System.out.println("\nGracias por usar el Simulador Financiero. ¡Hasta
pronto!");
        break;
    default:
        System.out.println("\nOpción no válida. Intenta de nuevo.");
    }
} catch (IllegalArgumentException e) {
    System.out.println("Error: " + e.getMessage());
}

} while (opcion != 4);

scanner.close();
}

// =====
// HELPERS DE LECTURA Y VALIDACIÓN
// =====

/**
 * Lee un double >= 0 por consola.
 */
private static double leerDoubleNoNegativo(Scanner scanner, String mensaje) {
    double valor = leerDouble(scanner, mensaje);
    while (valor < 0) {
        System.out.println("El valor no puede ser negativo.");
        valor = leerDouble(scanner, mensaje);
    }
    return valor;
}

/**
```

```
* Lee un double > 0 por consola.
*/
private static double leerDoublePositivo(Scanner scanner, String mensaje) {
    double valor = leerDouble(scanner, mensaje);
    while (valor <= 0) {
        System.out.println("El valor debe ser mayor que cero.");
        valor = leerDouble(scanner, mensaje);
    }
    return valor;
}

/**
 * Lee un double válido (cualquier valor real) por consola.
 */
private static double leerDouble(Scanner scanner, String mensaje) {
    System.out.print(mensaje);
    while (!scanner.hasNextDouble()) {
        System.out.print("Entrada inválida. " + mensaje);
        scanner.next();
    }
    double valor = scanner.nextDouble();
    scanner.nextLine(); // limpieza del buffer
    return valor;
}

/**
 * Lee un entero > 0 por consola.
 */
private static int leerEnteroPositivo(Scanner scanner, String mensaje) {
    System.out.print(mensaje);
    while (!scanner.hasNextInt()) {
        System.out.print("Entrada inválida. " + mensaje);
        scanner.next();
    }
    int valor = scanner.nextInt();
    scanner.nextLine(); // limpieza del buffer

    while (valor <= 0) {
        System.out.print("Debe ser mayor que cero. " + mensaje);
        while (!scanner.hasNextInt()) {
            System.out.print("Entrada inválida. " + mensaje);
            scanner.next();
        }
        valor = scanner.nextInt();
        scanner.nextLine();
    }
    return valor;
}
```

```
/**
 * Lee una tasa escrita por el usuario en porcentaje (ej. 5 para 5%)
 * y la convierte a decimal (0.05) antes de devolverla.
 * Valida que la tasa no sea negativa.
 */
private static double leerTasaComoPorcentaje(Scanner scanner, String mensaje) {
    double porcentaje = leerDouble(scanner, mensaje);
    while (porcentaje < 0) {
        System.out.println("La tasa no puede ser negativa.");
        porcentaje = leerDouble(scanner, mensaje);
    }
    return porcentaje / 100.0;
}
}
```

REFLECCION:

Honestamente me agrado el usar la ayuda de la ia, ya que no solo me dio la parte del código estuve preguntando prácticamente linea por linea el funcionamiento de lo entregado y fue altamente benéfico para comprender lo que hizo, como lo hizo y si se puede mejorar con otras funciones a futuro, siento que es como tener una plática con un profesional sobre el tema lo cual sirve mucho para ir adquiriendo nuevos conocimientos o reforzar los conocimientos ya obtenidos, en general me agrado usarlo aunque de repente entregaba algo que nada que ver pero con los conocimientos adquiridos fue fácil ir detectando esos errores e irlos corrigiendo.